

БИЛЕТИ №1 — Асосҳои омӯзиши машинӣ

1. Мафҳуми омӯзиши машинӣ, таърихи инкишофи он ва фарқияти он аз барномасозии классикӣ

Омӯзиши машинӣ (Machine Learning — ML) ин зерсоҳаи зеҳни сунъӣ аст, ки ба компютер имкон медиҳад аз маълумот худ ёд гирад, бидуни он ки барномасоз ҳама қоидаҳоро ба таври мушаххас нависад.

Таърих: Соли 1950 Алан Тюринг тести машҳури «тести Тюринг»-ро пешниҳод кард. Соли 1957 Фрэнк Розенблатт перцептронро ихтироъ кард. Дар солҳои 1980-1990 шабакаҳои нейронӣ ва алгоритмҳои дарахти қарор таҳия шуданд. Аз соли 2010 инчониб Deep Learning ва Big Data инқилоби воқеиро оварданд.

Фарқият аз барномасозии классикӣ: Дар барномасозии классикӣ барномасоз ҳама қоидаҳоро мушаххас мекунад (агар А, пас Б). Дар ML модел худааш қоидаҳоро аз маълумот меомӯзад. Мисол: барои фарқ кардани спам — дар классикӣ ҳар калима ба дастӣ нависем, дар ML модел аз ҳазорон мисол меомӯзад.

2. Намудҳои асосии омӯзиши машинӣ: supervised, unsupervised, reinforcement

Supervised Learning (омӯзиши назоратшаванда): Модел бо маълумоти дорои тамға (label) омӯзонида мешавад. Мисол: пешгӯии нархи хона, тасниф кардани почтаи спам. Алгоритмҳо: Linear Regression, Logistic Regression, SVM, Decision Tree.

Unsupervised Learning (омӯзиши беназорат): Маълумот тамға надорад, модел сохторро мустақилона меёбад. Мисол: гурӯҳбандии мизочон, кашфи аномалия. Алгоритмҳо: K-means, DBSCAN, PCA.

Reinforcement Learning (омӯзиш тавассути мукофот): Агент дар муҳит амал мекунад ва барои амалҳои дуруст мукофот мегирад. Мисол: бозиҳои компютерӣ, робот, мошини худрон.

3. Бо Python ва Pandas dataset таҳлил кунед, арзишҳои гумшударо муайян ва бартараф намоед

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.info()) # маълумот дар бораи сутунҳо
print(df.isnull().sum()) # шумораи арзишҳои гумшуда
# Усулҳои бартараф кардан:
df.dropna(inplace=True) # хатҳои дорои NaN-ро нест кунед
df.fillna(df.mean(), inplace=True) # бо миёна иваз кунед
```

Шарҳ: isnull() ячейкаҳои холиро ошкор мекунад. Агар маълумот кам бошад, fillna() беҳтар аст, зеро маълумотро гум намекунем.

4. Моделҳои оддии Linear Regression соzed ва пешгӯии параметрро иҷро кунед

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np

X = df[['feature1', 'feature2']]
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Шарҳ: Модел хатти рост меёбад, ки маълумотро беҳтарин тавсиф кунад. Коэффициентҳо (`model.coef_`) нишон медиҳанд, ки ҳар як омил то чӣ андоза ба натиҷа таъсир мерасонад.

5. Натиҷаи моделро бо MSE ҳисоб карда, таҳлил намоед

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_test, y_pred)
print(f'MSE: {mse:.4f}')
print(f'RMSE: {mse*0.5:.4f}')
```

Шарҳ: MSE (Mean Squared Error) миёнаи квадрати хатогиҳоро ҳисоб мекунад. Ҳарчанд MSE хурдтар бошад, модел дақиқтар аст. RMSE (решаи квадрати MSE) маъноашро дар воҳидҳои аслии дорад ва осонтар тафсир мешавад.

БИЛЕТИ №2 — Маълумот ва пешомодасозии он

1. Нақши маълумот дар омӯзиши мошинӣ ва таъсири сифат ва ҳаҷм ба дақиқии моделҳо

Маълумот бунёди асосии омӯзиши мошинӣ аст. Принципи «garbage in — garbage out» (зибал ворид кунӣ — зибал мегири) дар ML хеле муҳим аст.

Таъсири ҳаҷм: Маълумоти зиёд ба модел имкон медиҳад алгоритмҳои мураккабтарро меомӯзад ва генерализатсияи беҳтарро ба даст орад. Маълумоти кам боиси overfitting мегардад.

Таъсири сифат: Маълумоти нодуруст, такрорӣ ё нопурра ба натиҷаҳои нодуруст меоварад. Масалан, агар дар dataset-и кредитӣ маълумот фақат дар бораи мардони бой бошад, модел дар бораи занон ё камбағалон хатогии зиёд хоҳад дошт.

2. Равандҳои асосии оморасозии маълумот (data preprocessing)

- Тозакунӣ (Cleaning): нест кардани маълумоти такрорӣ, пур кардани арзишҳои гумшуда, бартараф кардани outlier-ҳо.
- Меъёрикунӣ (Normalization/Standardization): ба як миқёс овардани маълумот. StandardScaler: $(x - \text{mean}) / \text{std}$. MinMaxScaler: $(x - \text{min}) / (\text{max} - \text{min})$.
- Encoding: табдил додани маълумоти категориалӣ ба рақамӣ (Label Encoding, One-Hot Encoding).
- Feature Engineering: сохтани аломатҳои нав аз мавҷудаҳо.
- Тақсими маълумот: train (70-80%), validation (10%), test (10-20%).

3. Dataset тоза карда, One-Hot Encoding татбиқ намоед

```
import pandas as pd
df = pd.read_csv('data.csv')
# Тозакунӣ
df.drop_duplicates(inplace=True)
df.fillna(df.median(numeric_only=True), inplace=True)
# One-Hot Encoding барои сутунҳои категориалӣ
df_encoded = pd.get_dummies(df, columns=['city', 'gender'])
print(df_encoded.head())
```

Шарҳ: One-Hot Encoding ҳар арзиши категориалиро ба сутуни ҷудогона табдил медиҳад (0 ё 1). Масалан, 'шаҳр' бо Душанбе/Хуҷанд/Бохтар ба 3 сутуни дутоии алоҳида табдил меёбад.

4. Logistic Regression барои тасниф истифода баред ва натиҷаро таҳлил кунед

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

Шарҳ: Logistic Regression эҳтимоли тааллуқ ба синф (0 ё 1)-ро ҳисоб мекунад. Дар натиҷа мегӯяд: 'бемор дорои саратон аст бо эҳтимоли 87%'. Барои масъалаҳои дуии тасниф мувофиқ аст.

5. Accuracy ва Precision-ро ҳисоб карда, фарқияти онҳоро шарҳ диҳед

```
from sklearn.metrics import accuracy_score, precision_score
acc = accuracy_score(y_test, y_pred)
```

```
prec = precision_score(y_test, y_pred)
print(f'Accuracy: {acc:.4f}, Precision: {prec:.4f}')
```

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ — нисбати пешгӯиҳои дуруст ба ҳама пешгӯиҳо.

Precision = $TP / (TP + FP)$ — аз байни ҳамаи мусбатҳои пешгӯишуда, чанд дарсад воқеан мусбат буданд.

Фарқият: Accuracy умумӣ аст, Precision нишон медиҳад модел то чӣ андоза пешгӯиҳои «мусбат»-и нодуруст (false alarm) намекунад. Дар ташҳиси беморӣ Precision муҳимтар аст.

БИЛЕТИ №3 — Linear ва Logistic Regression

1. Алгоритми Linear Regression бо формула ва принципи кор

Linear Regression робитаи хатти байни омилҳо (X) ва натиҷа (y)-ро меомӯзад.

Формула: $y = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$

Дар ин ҷо w_0 — intercept (озод аъзо), $w_1 \dots w_n$ — вазнҳо (коэффисиентҳо).

Принципи кор: Алгоритм вазнҳоро тавре интихоб мекунад, ки хатогии умумӣ (MSE) кам бошад. Ин тавассути Gradient Descent ё Least Squares Method анҷом меёбад. Мисол: пешгӯии нархи хона бо назардошти масоҳат ва миқдори хонаҳо.

2. Фарқияти Linear ва Logistic Regression

Linear Regression: барои пешгӯии арзиши пайваستا (continuous) истифода мешавад. Натиҷа метавонад ҳар рақам бошад. Мисол: нарх, ҳарорат, даромад.

Logistic Regression: барои тасниф (classification) истифода мешавад. Натиҷа ба воситаи функсияи sigmoid ба байни 0 ва 1 маҳдуд мешавад. Мисол: спам/на спам, бемор/солим.

Функсияи sigmoid: $\sigma(z) = 1 / (1 + e^{(-z)})$

Хулоса: Linear → рақами пешгӯӣ, Logistic → эҳтимол ва синф.

3. Бо matplotlib хатти регрессияро тасвир кунед

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(X_train, y_train)
x_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_line = model.predict(x_line)

plt.scatter(X_test, y_test, color='blue', label='Маълумоти воқеӣ')
plt.plot(x_line, y_line, color='red', label='Хатти регрессия')
plt.xlabel('Омил (X)')
plt.ylabel('Натиҷа (y)')
plt.legend()
plt.savefig('regression.png')
plt.show()
```

4. Нархи хонаро бо модели Linear Regression пешгӯӣ намоед

```
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

data = fetch_california_housing()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)
# Пешгӯии нарх барои хонаи нав
new_house = [[3.5, 25, 5, 1.2, 500, 3, 37.5, -122.0]]
pred = model.predict(new_house)
```

```
print(f'Наpxи пешгӯишуда: ${pred[0]*100000:.0f}')
```

5. R^2 (коэффициенти муайянкунӣ)-ро ҳисоб кунед ва таҳлил намоед

```
from sklearn.metrics import r2_score  
r2 = r2_score(y_test, y_pred)  
print(f'R²: {r2:.4f}')
```

Маъно: R^2 нишон медиҳад, ки модел то чӣ андоза тағйирёбии маълумотро тавзеҳ медиҳад.

$R^2 = 1.0$ — модел комилан дақиқ аст

$R^2 = 0.8$ — модел 80% тағйирёбиро тавзеҳ медиҳад (хуб)

$R^2 = 0.3$ — модел танҳо 30% тавзеҳ медиҳад (заиф)

$R^2 < 0$ — модел аз миёна ҳам бадтар аст

БИЛЕТИ №4 — Decision Tree ва Random Forest

1. Decision Tree — шарҳ, афзалиятҳо ва камбудихо

Decision Tree (Дарахти қарор) — алгоритме, ки маълумотро бо пурсишҳои пайдарпай тақсим мекунад, монанди дарахт.

Принсип: Дар ҳар гирех (node) аломате интихоб мешавад, ки маълумотро беҳтарин тақсим кунад (бо Gini ё Entropy).

Афзалиятҳо: осон тафсир кардан ва визуализатсия, бо маълумоти рақамӣ ва категориалӣ кор мекунад, меъёрикунӣ лозим нест.

Камбудихо: ба overfitting хеле майл дорад (агар чуқурии дарахт маҳдуд карда нашавад), бо маълумоти каме нодуруст мешавад.

2. Random Forest чӣ гуна кор мекунад ва чаро дақиқтар аст?

Random Forest ансамбли дарахтони зиёди тасодуфӣ қарор аст. Ҳар дарахт дар зертаҷмин-и тасодуфӣ маълумот ва аломатҳо омӯзонида мешавад.

Принсипи Bagging: ҳар дарахт дар намунаи тасодуфӣ (bootstrap) омӯзонида мешавад. Барои тасниф, натиҷаи ниҳой бо раъйдихии аксариятӣ муайян мешавад.

Чаро дақиқтар аст: Дарахти ягона ба шум (noise) ва хатогиҳо ҳассос аст. Дарахтони зиёд ин хатогиҳоро «бо ҳам хунсо мекунад» — ки ин принсипи ensemble learning аст. Натиҷа variance паст, bias паст.

3. Модели Decision Tree созад ва feature importance муайян кунед

```
from sklearn.tree import DecisionTreeClassifier
import pandas as pd

dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train, y_train)

# Feature importance
fi = pd.Series(dt.feature_importances_, index=feature_names)
fi_sorted = fi.sort_values(ascending=False)
print(fi_sorted)
fi_sorted.plot(kind='bar')
```

Шарҳ: feature_importances_ нишон медиҳад, кадом аломат бештар ба тақсимои мусоидат кард. Арзишҳои калонтар = аломати муҳимтар.

4. Random Forest татбиқ намуда, натиҷаҳоро муқоиса кунед

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

print('Decision Tree accuracy:', accuracy_score(y_test, y_pred_dt))
print('Random Forest accuracy:', accuracy_score(y_test, y_pred_rf))
```

Одатан Random Forest аз Decision Tree 5-15% дақиқтар мебошад, зеро $n=100$ дарахт хатогиҳои ҳамдигарро ислоҳ мекунад.

5. Confusion Matrix ҳисоб карда, таҳлил намоед

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay  
cm = confusion_matrix(y_test, y_pred)  
print(cm)  
disp = ConfusionMatrixDisplay(cm)  
disp.plot()
```

Тафсир:

TP (True Positive) — дуруст мусбат пешгӯишуда

TN (True Negative) — дуруст манфӣ пешгӯишуда

FP (False Positive) — хатогии навъи I (шубҳаи бесабаб)

FN (False Negative) — хатогии навъи II (аз даст додани бемор)

БИЛЕТИ №5 — Кластербандӣ ва K-means

1. Мафҳуми clustering ва аҳамияти он дар таҳлили додаҳо

Clustering (Кластербандӣ) — ин усули омӯзиши беназорат аст, ки объектҳои шабеҳро ба гурӯҳҳо (кластерҳо) муттаҳид мекунад, бе он ки пешакӣ тамғаҳо дода шуда бошанд.

Аҳамият: Кластербандӣ дар маркетинг (сегментатсияи мизоҷон), тибб (гурӯҳбандии беморон), биология (дастаи генҳо), сайберамниятӣ (ошкор кардани anomaly) ва тавсияномаҳои истифода мешавад.

Мисол воқеӣ: Netflix мизоҷонро мувофиқи таърихи тамошо ба кластерҳо тақсим карда, тавсияҳои шахсишударо медиҳад.

2. Алгоритми K-means ва қадамҳои кори он

K-means — алгоритме, ки маълумотро ба K кластер тақсим мекунад.

Қадамҳо:

- Ба тасодуф K маркази ибтидоӣ (centroids) интихоб кунед
- Ҳар нуқтаро ба наздиктарин марказ тааллуқ диҳед (бо масофаи Евклид)
- Маркази ҳар кластерро аз нав ҳисоб кунед (миёна)
- Қадамҳои 2 ва 3 то яқоякунии марказҳо такрор кунед

Мураккабии вақт: $O(n * K * i)$, дар ин ҷо n — маълумот, K — кластер, i — такроршавӣ.

3. Dataset-ро ба гурӯҳҳо бо K-means тақсим кунед

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_scaled)

labels = kmeans.labels_
print('Тақсими кластерҳо:', pd.Series(labels).value_counts())
```

Шарҳ: Пеш аз K-means меъёрикунӣ (StandardScaler) ҳатмӣ аст, зеро алгоритм ба миқёси маълумот ҳассос аст.

4. Кластерҳоро дар график визуализатсия кунед

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1],
c=labels, cmap='viridis', alpha=0.6)
centers = kmeans.cluster_centers_

plt.scatter(centers[:, 0], centers[:, 1],
c='red', marker='X', s=200, label='Марказҳо')

plt.colorbar(scatter)
plt.legend()
plt.title('Натиҷаи K-means кластербандӣ')
plt.savefig('clusters.png')
```

```
plt.show()
```

5. Шумораи оптималии cluster-ро муайян намоед (Elbow method)

```
inertia = []  
for k in range(1, 11):  
    km = KMeans(n_clusters=k, random_state=42, n_init=10)  
    km.fit(X_scaled)  
    inertia.append(km.inertia_)  
  
plt.plot(range(1, 11), inertia, 'bo-')  
plt.xlabel('Шумораи кластерҳо (K)')  
plt.ylabel('Inertia')  
plt.title('Усули Elbow')  
plt.show()
```

Шарҳ: Дар графики Elbow нуқтаеро ёбед, ки пас аз он коҳиши inertia суст мешавад — ин K оптималӣ аст. Масалан, агар қачхат дар K=3 шикаст бихӯрад, K=3 беҳтарин аст.

БИЛЕТИ №6 — Алгоритми K-Nearest Neighbors (KNN)

1. Принципи кори KNN ва метрикаҳои масофа

KNN (K наздиктарин ҳамсоя) — алгоритми оддии тасниф ва регрессия. Барои объекти нав K наздиктарин ҳамсояро дар dataset меёбад ва ба синфи аксарият тааллуқ медиҳад.

Масофаи Евклид: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + \dots}$

Масофаи Манхэттен: $d = |x_1 - x_2| + |y_1 - y_2| + \dots$

Афзалиятҳо: омӯзиш лозим нест (lazy learner), барои маълумоти хурд хуб кор мекунад, ба маълумоти нав осон мутобиқ мешавад.

Камбудихо: дар dataset-и калон суҷ аст (ҳама масофаҳоро ҳисоб мекунад), ба маълумоти иловагӣ (irrelevant features) ҳассос аст.

2. Таъсири параметри K ба дақиқӣ — overfitting ва underfitting

K=1 (хеле хурд): Модел ба маълумоти омӯзиш комилан мутобиқ мешавад, аммо ба маълумоти нав суҷ кор мекунад — OVERFITTING. Ҳудуди тасниф хеле мураккаб ва ноҳамвор аст.

K=100 (хеле калон): Модел ҳамаро ба синфи аксарият тасниф мекунад — UNDERFITTING. Ҳудудҳо хеле содда ва нодақиқ мешаванд.

K оптималӣ: Одатан аз тавассути cross-validation муайян мешавад. Қоидаи тахминии: $K \approx \sqrt{n}$, дар ин ҷо n — шумораи намунаҳо. Адади тоқ бошад беҳтар аст (барои пешгирии баробарии овозҳо).

3. KNN барои тасниф бо Scikit-learn татбиқ намоед

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris = load_iris()

X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

4. Моделро бо K=3, 5, 7 омӯзонед ва муқоиса кунед

```
from sklearn.metrics import accuracy_score

for k in [3, 5, 7]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f'K={k}: Accuracy = {acc:.4f}')
```

Таҳлил: Натиҷаи ҳар K-ро муқоиса карда, K-ро, ки асрасу баландтар аст, интиҳоб кунед. Одатан дар dataset-и Iris K=5 ё K=7 беҳтарин нишон медиҳад.

5. Ассурасу ҳисоб карда, К беҳтаринро муайян намоед

```
accuracies = {}  
for k in range(1, 21):  
    knn = KNeighborsClassifier(n_neighbors=k)  
    knn.fit(X_train, y_train)  
    accuracies[k] = accuracy_score(y_test, knn.predict(X_test))  
  
best_k = max(accuracies, key=accuracies.get)  
print(f'Беҳтарин К: {best_k}, Accuracy: {accuracies[best_k]:.4f}')  
  
plt.plot(list(accuracies.keys()), list(accuracies.values()))  
plt.xlabel('K')  
plt.ylabel('Accuracy')  
plt.show()
```

БИЛЕТИ №7 — Шабакаҳои нейронӣ ва Deep Learning

1. Сохтори шабакаи нейронӣ — нейрон, қабатҳо, forward ва backpropagation

Шабакаи нейронӣ аз нейронҳо иборат аст. Ҳар нейрон маҷмӯи иштирок (weight)-ро ба вуруди худ татбиқ мекунад, онҳоро ҳамчун мезанад ва тавассути функсияи фаъолсозӣ мегузарад.

Қабатҳо: Input Layer (вурӯд), Hidden Layers (пинҳонӣ), Output Layer (баромад).

Функсияҳои фаъолсозӣ: ReLU ($f=\max(0,x)$), Sigmoid (0-1), Tanh (-1 то 1), Softmax (барои тасниф).

Forward propagation: маълумот аз вурӯд то баромад ҳисоб мешавад.

Backpropagation: ҳатогӣ аз баромад ба вурӯд бармегардад ва вазнҳо бо Gradient Descent навсозӣ мешаванд.

2. Deep Learning — фарқият аз ML классикӣ, афзалиятҳо ва маҳдудиятҳо

Deep Learning (DL) — зерсоҳаи ML бо шабакаҳои нейронии чуқур (бисёр қабатҳо).

Фарқият аз ML классикӣ: ML классикӣ feature engineering-и дастиро талаб мекунад. DL аломатҳоро худаш меомӯзад. ML классикӣ бо маълумоти кам ҳам кор мекунад, DL маълумоти зиёд ва қувваи ҳисоббарориро талаб мекунад.

Афзалиятҳо: Дар тасвир, садо ва матн беназир, натиҷаҳои бениҳоят дақиқ. Мисолҳо: ChatGPT, AlphaFold, Tesla Autopilot.

Маҳдудиятҳо: маълумоти зиёд лозим, ҳисоббарории гарон (GPU), ноданиш (black box).

3. Бо TensorFlow/PyTorch шабакаи нейронии оддӣ созед

```
import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid') # барои тасниф
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

print(model.summary())
```

4. Моделро омӯзонед, loss ва ассурасу таҳлил кунед, графики loss созед

```
history = model.fit(X_train, y_train,
                  epochs=50, batch_size=32,
                  validation_split=0.2,
                  verbose=1)

import matplotlib.pyplot as plt
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
```

```
plt.legend()
plt.title('Раванди омӯзиш')
plt.show()
```

Шарҳ: Агар `val_loss` аз `train_loss` хеле баланд бошад — `overfitting`. Агар ҳарду баланд бошанд — `underfitting`.

5. Таъсири шумораи қабатҳо ва нейронҳоро ба сифати модел таҳлил кунед

```
# Модели 1 қабата
model1 = keras.Sequential([keras.layers.Dense(16, activation='relu'), keras.layers.Dense(1)])

# Модели 2 қабата
model2 = keras.Sequential([keras.layers.Dense(64, activation='relu'),
keras.layers.Dense(32, activation='relu'),
keras.layers.Dense(1)])
```

Таҳлил: Модели 1 қабата ба мушкилоти содда мувофиқ аст. Қабатҳои зиёд имкон медиҳад мушкилоти мураккабтарро омӯзад, аммо хатари `overfitting` ва вақти омӯзиш зиёд мешавад. `Dropout` ва `BatchNormalization` барои пешгирии `overfitting` истифода бурда мешавад.

БИЛЕТИ №8 — Reinforcement Learning

1. Мафҳуми Reinforcement Learning ва унсурҳои асосии он

Reinforcement Learning (RL) — навъи омӯзише, ки агент тавассути санҷиш ва хато меомӯзад.

Унсурҳои асосӣ:

- Агент (Agent): субъекти омӯзанда ки қарор мегирад
- Муҳит (Environment): ҷаҳоне, ки агент дар он амал мекунад
- Ҳолат (State): вазъи ҷории муҳит
- Амал (Action): қарори агент дар ҳар қадам
- Мукофот (Reward): баҳои амали анҷомшуда (мусбат ё манфӣ)

Мисолҳо: бозии шатранҷ (AlphaGo), мошини худрон, робот, тавсияи реклама.

2. Фарқияти RL ва supervised learning, exploration vs exploitation

Supervised Learning: маълумоти тамғадор лозим, «муаллим» ҷавоби дурустро медиҳад. Мисол: шинохти акс.

Reinforcement Learning: тамға лозим нест, агент аз мукофоти муҳит меомӯзад. Мисол: омӯхтани бозӣ.

Дилеммаи Exploration vs Exploitation:

- Exploration: амалҳои нав санҷидан (барои дарёфти ҳалли беҳтар)
- Exploitation: аз донишҳои мавҷуда истифода бурдан (барои мукофоти фаврӣ)

Е-greedy стратегия: бо эҳтимоли ϵ тасодуфӣ амал кунад, бо $1-\epsilon$ беҳтарин амалро интихоб кунад.

3. Бо OpenAI Gym (CartPole) модели оддии RL татбиқ намоед

```
import gym
import numpy as np

env = gym.make('CartPole-v1')
n_episodes = 100
rewards_history = []

for episode in range(n_episodes):
    state = env.reset()[0]
    total_reward = 0

    for _ in range(500):
        # Стратегияи тасодуфӣ
        action = env.action_space.sample()
        next_state, reward, done, _, _ = env.step(action)

        total_reward += reward

    if done: break
    rewards_history.append(total_reward)
env.close()
```

4. Рафтори агентро дар ҷараёни омӯзиш таҳлил кунад

Агент дар аввал тасодуфӣ амал мекунад (reward паст). Бо гузашти вақт алгоритм (масалан, Q-learning ё DQN) арзиши ҳар амалро меомӯзад.

Q-learning формула:

$$Q(s,a) = Q(s,a) + \alpha * [r + \gamma * \max(Q(s',a')) - Q(s,a)]$$

Дар ин ҷо: α — суръати омӯзиш, γ — discount factor (арзиши мукофоти оянда), r — мукофоти фаврӣ.

Бо гузашти эпизодҳо reward мунтазам зиёд мешавад — ин нишонаи омӯзиши муваффақ аст.

5. Натиҷаи омӯзишро бо график (reward per episode) тасвир кунед

```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 4))
plt.plot(rewards_history, alpha=0.4, label='Reward ҳар эпизод')

# Миёнаи ҳаракаткунанда барои вузӯҳ
window = 10

moving_avg = np.convolve(rewards_history, np.ones(window)/window, mode='valid')

plt.plot(range(window-1, len(rewards_history)), moving_avg,
color='red', label=f'Миёнаи {window} эпизод')

plt.xlabel('Эпизод')
plt.ylabel('Мукофоти умумӣ')
plt.title('Раванди омӯзиши агент')
plt.legend()
plt.show()
```

БИЛЕТИ №9 — Метрикаҳои арзёбии модел

1. Accuracy, Precision, Recall, F1-score — формулаҳо ва маъно

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Ҳамаи пешгӯиҳои дурустро ба ҳама тақсим мекунад. Дар dataset-и нобаробар нодуруст мебошад!

$$\text{Precision} = TP / (TP + FP)$$

Аз пешгӯиҳои мусбат, чанд дарсад воқеан мусбат буданд? (дурустии мусбатҳо)

$$\text{Recall} = TP / (TP + FN)$$

Аз ҳамаи мусбатҳои воқеӣ, чанд дарсадро ёфтем? (пуррай)

$$\text{F1-score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Миёнаи гармоникии Precision ва Recall — барои dataset-и нобаробар беҳтарин метрика.

2. Confusion Matrix — шарҳ ва истифода барои таҳлили хатогиҳо

Confusion Matrix чадвалест, ки нишон медиҳад модел чандто TP, TN, FP ва FN дорад.

Мисол барои ташхиси беморӣ:

TP=90: беморонро дуруст мусбат муайян кард

TN=85: солимҳоро дуруст манфӣ муайян кард

FP=10: солимҳоро хатогӣ бемор гуфт (false alarm)

FN=15: беморонро хатогӣ солим гуфт (хатари хеле ҷиддӣ!)

Дар тиб FN хатарноктар аст: бемори воқеиро аз даст додан. Бинобар ин Recall муҳимтар аст.

3. Confusion Matrix барои Dataset ҳисоб кунед

```
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = RandomForestClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:')
print(cm)
print(classification_report(y_test, y_pred))
```

4. Precision ва Recall барои ҳар синф ҳисоб кунед

```
from sklearn.metrics import precision_score, recall_score

# Барои тасниф бисёрсинфа
precision = precision_score(y_test, y_pred, average=None)
recall = recall_score(y_test, y_pred, average=None)

for i, (p, r) in enumerate(zip(precision, recall)):
    print(f'Синфи {i}: Precision={p:.3f}, Recall={r:.3f}')
```

Шарҳ: average=None натиҷаро барои ҳар синф алоҳида бармегардонад. average='macro' миёнаи арифметикиро медиҳад, average='weighted' ба ҳаҷми ҳар синф вазн медиҳад.

5. F1-score ҳисоб карда, таҳлил ва хулоса кунед

```
from sklearn.metrics import f1_score

f1_per_class = f1_score(y_test, y_pred, average=None)
f1_macro = f1_score(y_test, y_pred, average='macro')
f1_weighted = f1_score(y_test, y_pred, average='weighted')

print(f'F1 барои ҳар синф: {f1_per_class}')
print(f'F1 macro: {f1_macro:.4f}')
print(f'F1 weighted: {f1_weighted:.4f}')
```

Хулоса: $F1 > 0.9$ — модел хуб. $F1 < 0.7$ — беҳбудӣ лозим. Агар Precision баланд ва Recall паст бошад — модел ҳушёр аст, аммо чизи зиёдеро аз даст медиҳад. Агар баръакс — модел ҳама чизро мегирад, аммо хатои дурӯғ зиёд аст.

БИЛЕТИ №10 — Самтҳои пешрафта ва Overfitting

1. Самтҳои пешрафтаи ML: NLP, Computer Vision, AutoML

NLP (Natural Language Processing — Коркарди забони табиӣ): машина матн ва сухани инсонро мефаҳмад. Мисолҳо: ChatGPT, тарҷума, таҳлили ҳиссиёт, чатботҳо. Асбобҳо: BERT, GPT, Transformer.

Computer Vision (Биниши компютерӣ): машина аксҳо ва видеоро фаҳмад. Мисолҳо: шинохти рӯй, таъхиси беморӣ аз рентген, мошини худрон. Асбобҳо: CNN, YOLO, ResNet.

AutoML (Омӯзиши мошинии худкор): ёбиши автоматии модел ва гиперпараметрҳо. Асбобҳо: Google AutoML, H2O.ai, Auto-sklearn.

2. Ояндаи ML ва таъсири он ба иқтисод, технология ва ҷомеа

Иқтисод: ML ҷойҳои корие, ки такрорӣ ва рутинӣ ҳастанд, иваз мекунад (ҳисобчиён, рондандагон, баъзе ҳуқуқшиносон). Аммо ҷойҳои корие, ки эҷодкорӣ ва ҳиссиёт лозим дорад, боқӣ мемонанд. Маҳсулнокии иқтисодӣ зиёд мешавад.

Технология: AGI (Зеҳни Умумии Сунъӣ) ҳадафи асосӣ аст. Quantum ML метавонад ҳисоббарориро ба таври инқилобӣ тезонад. Edge ML дар дастгоҳҳои кӯчак амал хоҳад кард.

Ҷомеа: нигаронӣ дар бораи махфияти маълумот, тасмимгирии алгоритмӣ дар одамон (кредит, кор, суд).

3. Модели дорои overfitting созад (Decision Tree бо depth калон)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Модели overfitting
dt_overfit = DecisionTreeClassifier(max_depth=None) # бидуни маҳдудият
dt_overfit.fit(X_train, y_train)

train_acc = accuracy_score(y_train, dt_overfit.predict(X_train))
test_acc = accuracy_score(y_test, dt_overfit.predict(X_test))

print(f'Train accuracy: {train_acc:.4f}') # ~1.00
print(f'Test accuracy: {test_acc:.4f}') # хеле паст
```

Нишонаи overfitting: train accuracy хеле баланд (0.99-1.0), test accuracy хеле паст (0.6-0.7).

4. Regularization ё маҳдуд кардани параметрҳо ро татбиқ кунед

```
# 1. Маҳдуд кардани чуқурии дарахт
dt_reg = DecisionTreeClassifier(max_depth=5, min_samples_split=10)

# 2. L2 Regularization дар Linear Regression
from sklearn.linear_model import Ridge
ridge = Ridge(alpha=1.0)

# 3. L1 Regularization
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.1)

# 4. Dropout дар шабакаи нейронӣ
# keras.layers.Dropout(0.3) — 30% нейронро дар омӯзиш хомӯш мекунад
```

Шарҳ: Regularization мураккабии моделро ҷарима мекунад, ки вазнҳо хеле калон нашаванд.

5. Cross-validation иҷро карда, натиҷаҳо ро муқоиса ва таҳлил намоед

```
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier

# Overfitting модел
dt_deep = DecisionTreeClassifier(max_depth=None)
scores_deep = cross_val_score(dt_deep, X, y, cv=5, scoring='accuracy')

# Regularized модел
dt_reg = DecisionTreeClassifier(max_depth=5)
scores_reg = cross_val_score(dt_reg, X, y, cv=5, scoring='accuracy')

print(f'Overfitting: {scores_deep.mean():.4f} ± {scores_deep.std():.4f}')
print(f'Regularized: {scores_reg.mean():.4f} ± {scores_reg.std():.4f}')
```

Шарҳ: CV-5 маълумотро ба 5 қисм мекунад, 5 бор модел меомӯзад ва натиҷаро миёнагирӣ мекунад. Дисперсияи (std) паст = модел устувор аст.

БИЛЕТИ №11 — Этика, Bias ва GridSearchCV

1. Этика дар омӯзиши машинӣ ва таъсири қарорҳои алгоритмӣ ба ҷомеа

Омӯзиши машинӣ ба ҳаёти одамон таъсири бевосита дорад: алгоритмҳо барои додани кредит, интихоби корманд, ҳукми судӣ ва ташхиси тиббӣ истифода мешаванд.

Масъалаҳои асосии этикӣ:

- Шаффофнок (Transparency): одамон бояд бидонанд, чаро алгоритм ин қарорро гирифт (XAI).
- Адолат (Fairness): алгоритм набояд нисбат ба гурӯҳҳои гуногун таъйиз кунад.
- Махфият (Privacy): маълумоти шахсии қорбарон бояд ҳифз шавад (GDPR).
- Масъулият: кӣ масъул аст, агар алгоритм хатоӣ кунад?

2. Мафҳуми bias дар ML ва сабабҳои пайдоиши он

Bias (Таъйиз) дар ML вақте пайдо мешавад, ки модел барои баъзе гурӯҳҳо хатогии зиёдтар мекунад.

Сабабҳои асосӣ:

- Bias дар маълумот: агар dataset фақат мардон дошта бошад, модел барои занон нодуруст қор мекунад. Мисол: Amazon дар соли 2018 системаи кадрӣ худро барои bias нисбат ба занон рад қард.
- Bias дар тамағузори: агар тамағузори ба гурӯҳи муайян бад бинанд, ин ба модел мегузарад.
- Confirmation bias: модел тасдиқи нуқтаи назари мавҷударо меомӯзад.

Усулҳои пешгирӣ: таҳлили маълумот, аудити алгоритм, маълумоти гуногунранг.

3. GridSearchCV барои Random Forest ё SVM истифода баред

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10]
}

rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(rf, param_grid, cv=5,
    scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)
```

4. Параметрҳои беҳтаринро муайян кунед

```
print('Беҳтарин параметрҳо:', grid_search.best_params_)
print('Беҳтарин accuracy:', grid_search.best_score_)

best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(X_test)
print('Test accuracy:', accuracy_score(y_test, y_pred_best))
```

Шарҳ: GridSearchCV ҳамаи комбинатсияҳои параметрҳоро санҷида, беҳтаринро бармегардонад. best_params_ — параметрҳое, ки баландтарин cv score доштанд. best_estimator_ — модели таълимдидашуда бо ин параметрҳо.

5. Натиҷаи модел пеш ва баъд аз optimization муқоиса кунед

```
from sklearn.metrics import accuracy_score, f1_score

# Модели ибтидоӣ
base_rf = RandomForestClassifier(random_state=42)
base_rf.fit(X_train, y_train)
base_acc = accuracy_score(y_test, base_rf.predict(X_test))

# Модели беҳбудёфта
opt_acc = accuracy_score(y_test, best_model.predict(X_test))

print(f'Пеш аз оптимизатсия: {base_acc:.4f}')
print(f'Баъд аз оптимизатсия: {opt_acc:.4f}')
print(f'Беҳбудӣ: +{(opt_acc-base_acc)*100:.2f}%')
```

Шарҳ: GridSearchCV одатан 1-5% беҳбудӣ меорад. Барои натиҷаи беҳтар RandomizedSearchCV ё Bayesian Optimization (optuna) истифода бурда мешавад.

БИЛЕТИ №12 — Data Cleaning ва китобхонаҳои Python

1. Раванди data cleaning ва аҳамияти он барои дақиқии моделҳо

Data Cleaning (Тозакунии маълумот) — раванди тайёр кардани маълумот барои таҳлил тавассути нест кардани хатогиҳо ва нопурраиҳо.

Марҳалаҳои асосӣ:

- Муайян кардани мушкилотҳо: NaN, дубликатҳо, outlier-ҳо, форматҳои нодуруст
- Коркарди NaN: пур кардан (fillna) ё нест кардан (dropna)
- Коркарди дубликатҳо: drop_duplicates()
- Коркарди outlier-ҳо: IQR метод, Z-score
- Мувофиқсозии форматҳо: санаҳо, сатрҳо, рақамҳо

Аҳамият: 80% вақти олим дар тозакунии маълумот сарф мешавад. Маълумоти тоза = модели дақиқ.

2. Китобхонаҳои асосии ML дар Python: NumPy, Pandas, Scikit-learn

NumPy: ҳисоббарории рақамии зуд бо массивҳои n-ченака. Вазифаҳо: array, matrix, algebra хаттӣ, тавлиди тасодуфӣ. Мисол: np.array, np.zeros, np.dot, np.linalg.solve

Pandas: таҳлили маълумоти ҷадвалӣ (DataFrame ва Series). Вазифаҳо: боргирии CSV/Excel, тозакунии филтр, гурӯҳбандӣ, якҷоякунии. Мисол: pd.read_csv, df.groupby, df.merge

Scikit-learn: алгоритмҳои ML. Вазифаҳо: оморасозии маълумот (Pipeline, StandardScaler), моделҳо (LR, RF, SVM, KNN), арзёбӣ (cross_val_score, GridSearchCV). API яхела: fit → predict → score.

3. Dataset бо Pandas боргирӣ ва таҳлил кунед (describe, info, visualization)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('data.csv')
print(df.info()) # намудҳои сутунҳо, NaN
print(df.describe()) # статистикаи тавсифӣ
print(df.shape) # ҳаҷм

# Visualization
df.hist(figsize=(12, 8))
plt.show()

# Матритсаи ҳамбастагӣ
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')

plt.show()
```

4. Бо Scikit-learn модели оддӣ созад

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Pipeline ин пайдарпаии амалиётро муттаҳид мекунад
pipeline = Pipeline([
```

```
('scaler', StandardScaler()),  
('model', LogisticRegression())  
])  
pipeline.fit(X_train, y_train)  
print('Accuracy:', pipeline.score(X_test, y_test))
```

5. Натиҷаҳо бо график визуализатсия кунед

```
import matplotlib.pyplot as plt  
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay  
  
# Confusion Matrix  
ConfusionMatrixDisplay.from_estimator(pipeline, X_test, y_test)  
  
plt.title('Confusion Matrix')  
plt.show()  
  
# ROC Curve (барои тасниф)  
RocCurveDisplay.from_estimator(pipeline, X_test, y_test)  
  
plt.title('ROC Curve')  
plt.show()
```

Шарҳ: ROC Curve нишон медиҳад, ки чӣ гуна модел TPR ва FPR-ро мувозинат мекунад. Масоҳати зери қачхат (AUC) = 1 беҳтарин, AUC = 0.5 тасодуфӣ аст.

БИЛЕТИ №13 — Overfitting, Underfitting ва Regularization

1. Мафҳумҳои overfitting ва underfitting бо мисолҳо

Overfitting (Бисёр мутобиқ шудан): Модел маълумоти омӯзиширо аз бар мекунад, аз ҷумла шуми ғайримухим. Дар train accuracy хеле баланд (0.99), дар test accuracy паст (0.70). Мисол: Decision Tree бо `max_depth=None`.

Underfitting (Кам мутобиқ шудан): Модел хеле содда аст ва ҳатто маълумоти омӯзиширо дуруст тавсиф карда наметавонад. Ҳам train ҳам test accuracy паст. Мисол: Decision Tree бо `max_depth=1`.

Нуктаи мувозинат (Bias-Variance Tradeoff): Беҳтарин модел дар байни overfitting ва underfitting ҷойгир аст. Validation curve ин нуқтаро нишон медиҳад.

2. Усулҳои пешгирии overfitting

1. Regularization (L1/L2): ба функсияи ҳатогӣ ҷарима илова мекунад барои вазнҳои калон.
2. Dropout (дар NN): дар ҳар қадами омӯзиш нейронҳои тасодуфӣ хомӯш мешаванд.
3. Cross-validation: арзёбии модел дар қисмҳои гуногуни маълумот.
4. Early Stopping: омӯзиш вақте қатъ мешавад, ки `val_loss` зиёд шудан гирад.
5. Data Augmentation: зиёд кардани маълумоти омӯзиш тавассути тағйирот (чархондан, кашидан).
6. Ensemble Methods: якҷо кардани моделҳои зиёд (Random Forest, Boosting).
7. Маҳдуд кардани мураккабии модел: `max_depth`, `min_samples_leaf`.

3. Dataset аз Kaggle интиҳоб кунед ва тозакунии маълумот иҷро кунед

```
# Мисол: Titanic dataset
import pandas as pd

df = pd.read_csv('titanic.csv')
print(df.isnull().sum()) # NaN-ҳоро муайян кунед

# Коркарди арзишҳои гумшуда
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
df.drop(columns=['Cabin'], inplace=True) # 77% NaN

# Коркарди outlier-ҳо бо IQR
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['Fare'] >= Q1-1.5*IQR) & (df['Fare'] <= Q3+1.5*IQR)]

df.drop_duplicates(inplace=True)
print('Баъд аз тозакуни:', df.shape)
```

4. Модел созед ва натиҷаҳоро таҳлил намоед

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Encoding
le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex'])
```

```

df['Embarked'] = le.fit_transform(df['Embarked'].astype(str))

features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']

X = df[features]
y = df['Survived']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = GradientBoostingClassifier(n_estimators=100, max_depth=3)
model.fit(X_train, y_train)
print('Accuracy:', model.score(X_test, y_test))

```

5. Тозакунни маълумот ичро кунед ва модели ниҳой арзёбӣ намоед

```

from sklearn.model_selection import cross_val_score
from sklearn.metrics import classification_report

# Cross-validation
cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f'CV Accuracy: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}')

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))

# Муҳимии аломатҳо
import pandas as pd
fi = pd.Series(model.feature_importances_, index=features)
fi.sort_values().plot(kind='barh')
plt.title('Feature Importance')
plt.show()

```

Хулоса: Тозакунни маълумот асраҷу-ро одатан 5-15% зиёд мекунад. Cross-validation таъмин мекунад, ки натиҷа воқеан боэтимод аст.